

GPT-6.1 Sol across every effort level

VulcanBench Frontier v4 | Code quality protocol v3.18 | October 2026

Abstract

GPT-6.1 Sol ran the 23-task VulcanBench Frontier v4 suite through Codex CLI 0.159.0 on a ChatGPT Pro subscription on September 29 to 30, 2026, once per task at each of five effort levels, Low to Max, 115 runs in all. Code quality carries 33% of the combined score and is judged for a named human reader by Muse Spark 1.3 (Meta) and Grok 4.6 (xAI) under the same frozen protocol as the other Frontier v4 reports, with a ground-truth intent-recovery probe. The combined score is 86.22 at Low, 86.44 at Medium, 88.23 at High, 88.78 at Extra-high and 88.35 at Max, a flat curve. GPT-6.1 Sol passes 20, 22, 23, 23 and 23 of 23 tasks from Low to Max and fixes 225, 228 and then all 231 of the 231 hidden behaviours tested. Code quality runs from 70.25 to 74.99. Cost per task runs from \$0.31 at Low to \$0.46 at Max; the whole sweep prices at \$44.33, \$0.39 per task.

Every run finished inside the flat 3-hour task bound and every run carries a published Code quality score, so every cell is judged on 23 runs. One run, Medium paddockcore, is scored from Muse Spark 1.3 alone because Grok 4.6's review of it had no valid response. From High up GPT-6.1 Sol passes every hidden test, so Frontier v4 no longer separates its top levels on correctness. It leads GPT-6 Sol and GPT-5.6 Sol at every level on combined score, tasks passed and cost.

Table 1. Combined score, Code quality, runtime and tasks passed by effort

Effort	Combined (33%)	SE	Combined (20%)	Code quality	Min/task	Judged	Passed
Low	86.22	0.82	87.93	70.79	6.9	23	20/23
Medium	86.44	0.78	87.95	70.25	10.6	23	22/23
High	88.23	0.40	89.31	73.67	10.2	23	23/23
Extra-high	88.78	0.40	89.76	74.99	15.4	23	23/23
Max	88.35	0.33	89.46	73.80	14.5	23	23/23

Combined (33%) = 0.50 functional + 0.085 lint and complexity + 0.085 security + 0.33 Code quality. Combined (20%) applies the prior 50/15/15/20 profile to the same Code quality scores. SE is one sample standard error across the cell's tasks, not judge uncertainty or a significance test. Runtime is solver wall-clock per task; judging is excluded.

The Code quality protocol

Two failure modes motivated the reviewed score. The lint and complexity metric rewards compression: the maintainability index carries a lines-of-code term and per-function complexity stays low when each dense line does something different, so five statements on a line with names like `x` and `k` can outscore the same logic written for a person. And a large model reading compressed code pays almost nothing to parse it; when a frontier model was calibrated as a readability judge, it rated a squashed six-line function the same as its formatted copy. Since September 7, 2026 Code quality carries 33% of the combined score and lint and complexity and security carry 8.5% each. The prior profile is reported beside the new one on every table.

Protocol v3.18 is the same protocol applied to this population. Nothing in the rubric, controls, quirk keys, gates, repeats, seed, weights or judges changed from v3.7; both judges retook the calibration exam under v3.18 before any counted call. The population froze on September 30 with 115 rows, none missing and none excluded. The frozen summary publishes a submission from the panels with a valid review; that rule applied once, described under Results in detail.

The rubric

Every judge receives the same instructions, frozen by hash before any review. The prompt names the reader it scores for: an engineer who has never seen the code, reads it top to bottom without running it, and must make a correct change in one sitting. It tells the judge that its own ease at parsing dense code is not evidence of readability. Six dimensions are scored 0 to 4 in half steps against written anchors, each with an exact excerpt and a concrete consequence for that reader.

Sub-score	Dimension	What is assessed
Human readability	Naming	Identifiers say what things are in the task's domain
Human readability	Presentation	Statements per line, nesting, function length, how much the reader must hold at once
Human readability	Intent	Constants, thresholds and quirks explained by names or accurate comments that say why
Maintainability	Structure	Coherent responsibilities, no duplicated policy, no abstraction beyond the task's scale
Maintainability	Changeability	A named plausible change, every edit site traced, scored on locality and safety
Maintainability	Verifiability	Explicit state, failures that name what went wrong, seems to test one rule alone

The layers under the 33 points

Reviewed panel, 24 points. The six-dimension rubric above, averaged across the judges with a valid review. **Intent recovery, 9 points.** Every task in this suite is a rewrite of a retired binary whose real behaviour departs from its written specification in documented ways. The judge sees only the specification and the code, never the issue text, and lists where the code departs from the specification; a separate call matches that list against the frozen answer key. The denominator is the quirks the submission actually passed tests for, so a functional failure is not punished twice; a submission that passed none would have no intent-recovery score and its Code quality would be the reviewed score alone, which no GPT-6.1 Sol run needed. **Measured maintenance, designed for 12 points, not yet built.** Until it exists the pre-registered split above is in force.

Judges and calibration

The scored panel is Muse Spark 1.3 through the Muse CLI on its Standard tier, which does not train on prompts, and Grok 4.6 through the Cursor CLI at medium effort, the v3.7 pair under the same pinned binaries. Neither lab has a model on this board, so neither judge grades a relative. Every session is fresh, tools are disabled, the workspace is empty, model identity is checked per call from the CLI's own records, and solver labels are withheld.

The calibration exam

Before scoring a single submission each judge reviews ten held-out programs that implement the same ledger specification: clear, compressed, compressed then auto-formatted, verbose with duplicated policy, needlessly abstracted, misleadingly commented, narrated with a comment on every line, a documented legacy quirk, an embedded instruction to give full marks, and hidden module state. Each program is reviewed five times in a seeded order. Twenty gates fixed in advance check that the judge sees the construct; a judge may miss at most one gate by at most half a point.

Table 2. Calibration verdicts

Judge	Protocol	Calls	Result	Allowance	Failing gates
Muse Spark 1.3	code-quality-maintenance-v3.18	80	passed	not used	none
Grok 4.6	code-quality-maintenance-v3.18	80	passed	not used	none

Both judges passed every gate with no allowance used, as under v3.17.

Table 3. Control means on five of the ten calibration programs

Judge	Control	Naming	Present.	Intent	Struct.	Change.	Verif.
Muse	clear	3.90	3.90	3.30	4.00	4.00	3.80
Muse	compressed	1.10	1.70	1.00	3.50	2.60	2.90
Muse	formatted	1.10	3.30	1.20	3.90	2.60	3.00
Muse	misleading comments	3.90	4.00	0.50	3.90	2.40	3.30
Muse	hidden state	2.60	2.80	1.90	1.80	2.60	1.90
Grok	clear	3.60	3.50	3.40	4.00	4.00	3.50
Grok	compressed	1.50	1.80	1.60	3.20	2.70	3.00
Grok	formatted	1.50	2.40	1.60	3.30	2.90	3.00
Grok	misleading comments	3.50	3.30	1.00	3.40	2.20	3.00
Grok	hidden state	2.90	3.00	2.50	2.10	3.20	2.10

Full gate values and control means are in calibration.json.

Results in detail

Table 4. Code quality components at each effort level

Component	Low	Medium	High	Extra-high	Max
Code quality	70.79	70.25	73.67	74.99	73.80
Human readability	61.5	60.9	64.8	66.6	64.8
Maintainability	72.7	73.4	76.6	77.7	77.6
Intent recovery	80.6	78.6	81.6	82.6	80.7
Reviewed score	67.1	67.1	70.7	72.1	71.2
Rated by Muse Spark 1.3	65.6	64.7	69.9	71.2	69.2
Rated by Grok 4.6	68.7	70.3	71.5	73.1	73.2
Standard error of Code quality	1.22	1.25	1.18	1.20	0.85
Hidden behaviours fixed of 231	225	228	231	231	231

Means cover 23 runs per cell; Grok 4.6's Medium mean covers the 22 runs it reviewed validly. Grok is the higher rater at every level.

Table 5. Four factors by effort, mean score out of 100

Effort	Functional	Lint, complexity	Security	Code quality
Low	97.17	81.85	86.09	70.79
Medium	98.70	81.49	82.17	70.25
High	100.00	81.42	82.39	73.67
Extra-high	100.00	80.76	84.35	74.99
Max	100.00	80.95	83.70	73.80

The one-panel Medium row

On paddockcore at Medium, Grok 4.6's primary review had no valid response after the protocol's single retry: both attempts quoted `self.standing[pony] += 2` as evidence where the code reads `self.standing[parts[1]] += 2`, a changed token that no recovery rule accepts. Following the owner's v3.14 decision on the same case, the rule `invalidate_unrecoverable_primary` marked the call invalid, and the frozen summary scores the run from Muse Spark 1.3 alone: reviewed score 54.17, intent recovery 53.57, Code quality 54.00, combined 80.76. The run passed its tests. Grok's probe answer for that run is not used and not published. The harness score card's intent-recovery row at Medium (78.4) averages both judges' probes on that run; the published Medium figure is 78.59.

Operator record

On high codeccore, Muse Spark 1.3's first probe attempt wrote an invalid JSON escape and its second quoted a code line with the string escapes decoded into control characters. By owner decision the new rule `recover_escaped_excerpts` respelled that excerpt with the source's escapes, after which it had to be verbatim; attempt 2 passed every other check and was selected. Three runs (low payrollcore, medium lodgecore and low cellarcore) add test fixtures the saved text patch cannot re-apply, so their judging evidence was rebuilt from the saved workspace's index diff after checking it against the run's patch; only fixtures differ. Each judge made 440 counted calls (80 in calibration, 115 primary reviews, 5 repeats, 10 pairwise checks, 115 intent probes, 115 answer-key matches); Muse needed a second attempt on eight and Grok on nine. Neither judge produced a reviewer fallback.

Cost and tokens across effort levels

All 115 runs priced from their Codex receipts at list API rates, cache-aware, solver inference only, judging excluded. GPT-6.1 Sol ran on a ChatGPT Pro subscription, so these are API-equivalent estimates rather than bills. Low is the cheapest and fastest level. Medium uses the most tokens per task and High the fewest, so High costs less than Medium; Max is the most expensive level.

Table 6. API-equivalent cost, raw tokens and runtime by effort

Effort	Runs	\$/task	Level total	Tokens/task	Min/task
Low	23	\$0.31	\$7.04	1.24M	6.9
Medium	23	\$0.40	\$9.17	1.62M	10.6
High	23	\$0.33	\$7.59	0.91M	10.2
Extra-high	23	\$0.43	\$10.00	1.12M	15.4
Max	23	\$0.46	\$10.54	1.04M	14.5
Full sweep	115	\$0.39	\$44.33	136M	22.0 h

Rates checked 2026-09-29: GPT-6.1 Sol at \$2.00 input, \$0.10 cached input and \$10.00 output per million tokens. Codex receipts report input, cached input, output and reasoning tokens per run, so cached input is billed at the cache-read rate and the rest at the standard rate. The sweep stamped each run at these rates at run time; the export recomputed every run from its receipt and matched every stamp. Tokens are raw solver totals including cache reads.

Limitations recorded with the estimates

- Codex receipts report input, cached input, output and reasoning tokens per run; cached input is billed at the cache-read rate and the rest of the input at the standard rate. Cache writes are free on this API and are not modelled.
- Standard tier, short-context rates. OpenAI bills prompts over 272K input tokens at a higher rate, but Codex keeps each request within its 272K context window, so no long-context premium applies and none is modelled.
- No Batch, Flex, Fast or priority pricing is applied.
- The estimate covers the solver's exposed receipts, not an independently observed API invoice.
- The sweep stamped each run at run time at the published GPT-6.1 Sol list rates; the export recomputes every run from its receipt and matched every stamp.

Three generations of Sol

GPT-6 Sol and GPT-5.6 Sol ran the same 23 tasks through Codex in September and were judged by the same judges under v3.17 and v3.7 (GPT-6 Sol at Medium and GPT-5.6 Sol at Max judged on 22 of 23). GPT-6.1 Sol leads both at every effort level on combined score, tasks passed and cost per task, and its Code quality is higher at every level.

Table 7. GPT-6.1 Sol, GPT-6 Sol and GPT-5.6 Sol by effort

Effort	Combined	Passed of 23	\$/task	Min/task
Low	86.22 / 67.62 / 69.05	20 / 4 / 6	\$0.31 / \$1.02 / \$1.36	6.9 / 13.0 / 9.8
Medium	86.44 / 80.60 / 82.56	22 / 13 / 12	\$0.40 / \$1.76 / \$1.72	10.6 / 17.5 / 11.1
High	88.23 / 82.83 / 85.76	23 / 15 / 20	\$0.33 / \$2.33 / \$2.05	10.2 / 23.8 / 12.0
Extra-high	88.78 / 85.94 / 86.47	23 / 18 / 21	\$0.43 / \$2.00 / \$1.61	15.4 / 20.9 / 10.6
Max	88.35 / 86.82 / 87.18	23 / 19 / 22	\$0.46 / \$2.52 / \$1.77	14.5 / 24.5 / 11.6

Each cell reads GPT-6.1 Sol / GPT-6 Sol / GPT-5.6 Sol. Combined scores are over judged runs; pass counts, cost and minutes cover all 23 runs per cell. Each model is priced at its own list rates. The sweeps ran on different Codex CLI versions (0.159.0, 0.155.0 and 0.153.4). GPT-6.1 Sol is slower than GPT-5.6 Sol at Extra-high and Max.

On the Frontier v4 board

GPT-6.1 Sol's best level, Extra-high at 88.78, ranks 13 of the board's 49 columns, behind 12 columns of Fable 5.1, Opus 5.5, GPT-6 Astra and GPT-5.6 Terra. Fable 5.1 and GPT-6 Astra at Max also pass all 23 tasks, and Opus 5.5 at High all 22 of its judged runs, so the gap is in the judged and scanned factors, mainly Code quality (74.99 against 82.43 for Fable 5.1 at Max and 76.26 for GPT-6 Astra at Max), and against Opus 5.5 also its security score. Of the 16 columns scoring 88 or more, GPT-6.1 Sol's three are the cheapest per task.

Perfect hidden tests from High up

From High to Max every task passes every hidden test, so Frontier v4 no longer separates GPT-6.1 Sol's top levels on correctness; Code quality, the lint and security scans, and cost still do. The integrity audit is clean on all 115 runs (no web access, no benchmark-data or answer-key paths), and OpenAI gives GPT-6.1 Sol a knowledge cutoff of April 30, 2026, before the tasks were built in August 2026. The suite charter's saturation-pruning rule calls for re-gating the suite when a new frontier generation lands; that is a planned follow-up.

How it was run

Codex CLI 0.159.0. Codex 0.155.0, 0.157.0 and 0.158.0 refuse GPT-6.1 Sol on a ChatGPT account before any work; 0.159.0 is the first release that serves it and runs from its own install for this column only. **The sweep overlapped another model's judging.** The solver sweep ran September 29, 19:18 PDT, to September 30, 17:21 PDT; at the owner's request its first 11.5 hours, to September 30, 06:51 PDT, ran alongside GPT-6 Sol's v3.17 judging on the same machine, covering every Low, Medium and High run and the first five Extra-high runs, which matters for the wall-clock figures. **Judging** ran September 30, 20:58 PDT, to October 1, 06:01 PDT, and 06:04 to 12:52 PDT after the Grok finding on page 4, with no solver sweep running. **One attempt per task and level;** no run was retried, and effort labels are Codex's own.

Evidence and reproduction

Public files: `runs.json` and `runs.csv` with every run's factors, hidden behaviours fixed, integrity verdicts and each scored judge's six-dimension sub-scores and intent recovery; `groups.json`; `calibration.json` with every gate value and control mean; `judge-protocols.json` with the exact rubric, system text, probe and match instructions, schemas, weights, the one-panel row, the escaping recovery and the three evidence rebuilds; `economics.json` with cost and token aggregates, the rate table and pricing limitations; and `provenance.json` with source hashes and limits.

<https://github.com/morganlinton/VulcanBenchCOM/tree/main/assets/data/swe-v4-gpt61-sol-v318>

Withheld: raw prompts and responses; submitted patches and reconstructed sources; quirk answer keys (they describe hidden-test behaviour); reviewer session identifiers and usage receipts; the unused Grok probe answer on medium paddockcore.

Recompute the published numbers

Each run's Code quality is $(0.24 \times \text{reviewed score} + 0.09 \times \text{intent recovery}) / 0.33$, where both terms are the equal mean of the judges with a valid review (both, except Muse Spark 1.3 alone on Medium paddockcore). The combined score is $100 \times (0.50 F + 0.085 Q + 0.085 S + 0.33 C / 100)$ with F, Q, S on a 0 to 1 scale. Group means weight the cell's tasks equally. The export script recomputes every row from the frozen summary, re-prices every run from its receipt, asserts that the one invalid judge call is the Grok review of Medium paddockcore, checks the figures against the harness card tables, and refuses to write if any value differs.

What this report does not claim

No human rated anything; the scores are model judgment for a human reader, not human validation. Standard errors describe task sampling only. The measured-maintenance layer is unbuilt, so the 33 points are 24 reviewed plus 9 intent recovery. Runs were not repeated. Hash checks bind the export to frozen files; they do not prove that judges were unbiased or that no training overlap exists.

Appendix. Per task at Low, Medium and Max

Task	Low CQ	Max CQ	Low combined	Medium combined	Max combined
blendcore	67.0	69.9	85.64	88.12	86.23
cellarcore	58.3	75.2	84.65	84.94	88.76
codeccore	81.0	75.3	90.61	88.66	88.49
depotcore	67.2	70.5	85.65	84.11	86.55
freightcore	78.0	81.8	91.45	88.06	90.39
granarycore	69.5	70.3	85.64	82.46	87.05
hedgcore	74.9	69.5	87.63	88.89	87.42
lodgcore	66.9	74.2	86.57	86.07	88.56
matchcore	76.5	76.5	90.96	88.09	87.78
metercore	71.2	76.8	87.32	87.04	88.91
pacecore	77.4	78.5	89.30	88.34	90.50
paddockcore	62.2	66.0	83.42	80.76*	84.57
payrollcore	66.6	79.2	86.27	87.49	90.85
qlite	73.5	80.3	88.87	90.35	91.10
queuecore	64.7	68.5	87.04	88.95	87.24
quotacore	75.0	74.2	89.56	88.60	89.39
reflowcore	74.2	73.3	77.87	87.51	88.82
schedcore	75.1	71.6	89.20	88.32	87.73
settlecore	73.4	76.5	77.81	86.87	88.42
snapcore	71.8	74.8	77.93	72.50	89.31
stampcore	65.7	73.2	86.05	88.02	88.92
tallycore	62.6	72.1	84.48	87.65	88.45
vaultcore	75.3	69.2	89.17	86.36	86.55

Per-task values from runs.json. Task names are shortened for width. * scored from Muse Spark 1.3 alone, as described on page 4.

Source hashes

Frozen artifact	SHA-256
v3.18/summary.json	288f844bc09e87334d6ebef0d803dc91...
v3.18/protocol.json	c7dbb6586e59d80433f9e96210219c47...
v3.18/private-manifest.json	ba6f816ffbeb51a04fdf9ec58f30569...
v3.18/calibration-muse.json	025ad9a4b708e8527fb4295594a5cfbc...
v3.18/calibration-grok.json	de38cfcd3bf000644f641ff03697c94...
calls/grok/primary/submission-054/operator-invalid.json	06429cac0ec95468a37ec529e15e4ff5...
calls/muse/probe/submission-083/selected.json	45e77ffcbb9ffa0d4f3edf765da1cd5c...
calls/muse/probe/submission-083/operator-invalid.superseded.json	3a34f640713f6080a5a749c4784ed814...
reconstruction/legacy-cellarcore-binary-parity-bdca534c.json	2b30fd1ebf5466f512936f21ed58d7e5...
reconstruction/legacy-lodgcore-binary-parity-08b754d0.json	77d2dd8fcc657511678a20f5614c2c0c...
reconstruction/legacy-payrollcore-binary-parity-51181245.json	de29fabfa60b94d251ff246d05eedc12...
comparison.json	7359debe2d78a801b894037ca63c599a...
gpt61-sol-v318-efforts.csv	a902a7a8294018cb17a5d5c5fc5cdf3b...
gpt61-sol-v318-economics-efforts.csv	d7606e95b149012b29cc06e86ec25a80...
sol-family-combined-efforts.csv	1a78547e417cb322edcb050bab8c612c...

VulcanBench Frontier v4: GPT-6.1 Sol across effort levels

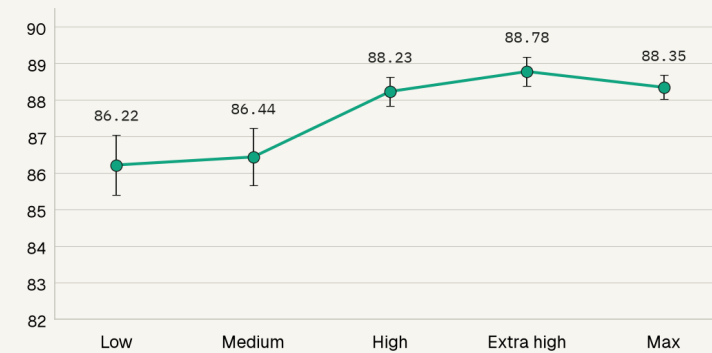
Combined score and runtime at every effort level, 23 tasks per effort. Code quality judged by Muse Spark 1.3 and Grok 4.6.

● **GPT-6.1 Sol** • Codex 0.159.0

n=23 at every effort

Combined score

/100 • higher is better • focused scale



Mean runtime

Minutes per task • lower is better

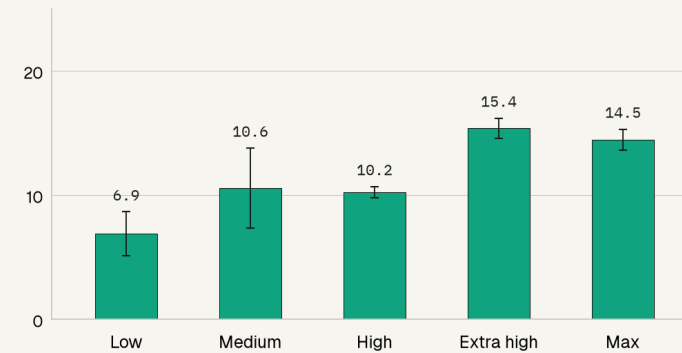


Table 1 | Code quality at each effort level

Component	Low	Medium	High	Extra high	Max
/100, mean of both judges					
Code quality	70.79	70.25	73.67	74.99	73.80
Human readability	61.5	60.9	64.8	66.6	64.8
Maintainability	72.7	73.4	76.6	77.7	77.6
Intent recovery	80.6	78.4	81.6	82.6	80.7
Rated by Muse Spark 1.3	65.6	64.7	69.9	71.2	69.2
Rated by Grok 4.6	68.7	70.3	71.5	73.1	73.2
Standard error of Code quality	1.22	1.25	1.18	1.20	0.85

All 115 runs finished inside the flat 3-hour task bound (longest 80 minutes); every cell is judged on 23 runs.

Medium paddockcore has no valid Grok 4.6 review (both attempts quoted a changed line), so, as the protocol does for any such run, its Code quality is Muse Spark 1.3's alone (reviewed layer and intent recovery). High codecore's Muse probe was recovered after a JSON escaping slip. Ran on Codex CLI 0.159.0, the first release that serves GPT-6.1 Sol on a ChatGPT plan.

VulcanBench Frontier v4: GPT-6.1 Sol across effort levels

API-equivalent cost and token use at every effort level. All 115 runs of the sweep; solver inference only, judging excluded.

● **GPT-6.1 Sol · Codex 0.159.0**

n=23 at every effort

API-equivalent cost

USD per task at list prices · lower is better



Tokens per task

Millions of raw tokens, cache reads included · lower is better

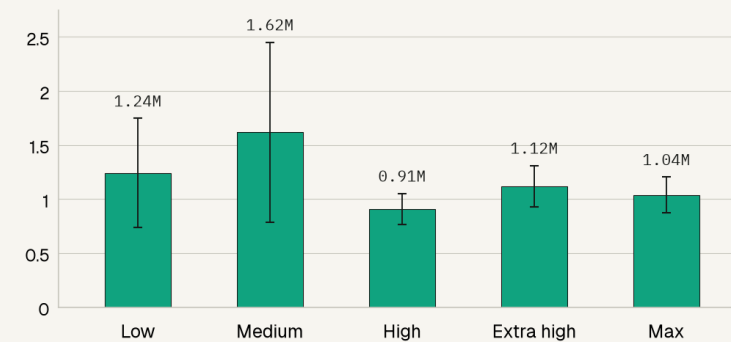


Table 1 | Cost, tokens and runtime at each effort level

Effort	Runs	\$/task	Level total API-equivalent, list prices	Tokens/task raw, millions	Minutes/task solver wall-clock
Low	23	\$0.31	\$7.04	1.24M	6.9
Medium	23	\$0.40	\$9.17	1.62M	10.6
High	23	\$0.33	\$7.59	0.91M	10.2
Extra high	23	\$0.43	\$10.00	1.12M	15.4
Max	23	\$0.46	\$10.54	1.04M	14.5
Full sweep	115	\$0.39	\$44.33	136M	22.0 h

USD at list prices checked 2026-09-29, cache-aware (GPT-6.1 Sol \$2.00 input, \$0.10 cached input, \$10.00 output per million), solver inference only; subscription bills differ.

All 115 runs are priced; none reached the 3-hour task bound.

Whiskers are one task standard error. Tokens are raw solver totals including cache reads.

VulcanBench Frontier v4: three generations of Sol

Combined score and cost per task at every effort level, 23 tasks per effort. Code quality judged by Muse Spark 1.3 and Grok 4.6.

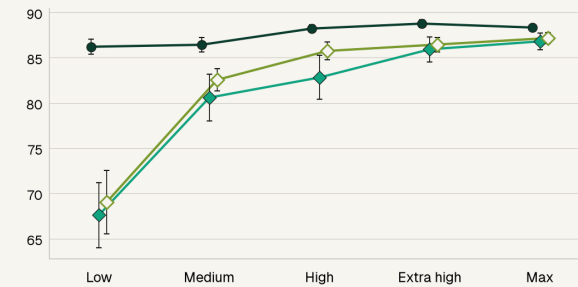
● GPT-6.1 Sol · Codex 0.159.0

◆ GPT-6 Sol · Codex 0.155.0

◇ GPT-5.6 Sol · Codex

Combined score

/100 · higher is better



API-equivalent cost

USD per priced task at list prices · log scale · lower is better

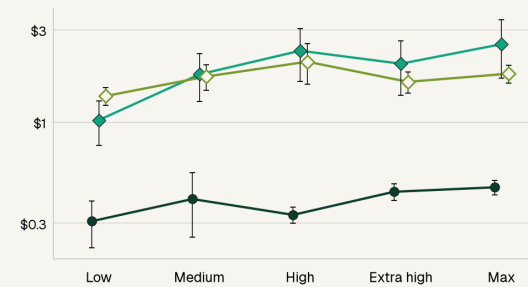


Table 1 | Each model at each effort level

Model and measure		Low	Medium	High	Extra high	Max
● GPT-6.1 Sol	Combined score	86.22	86.44	88.23	88.78	88.35
	Tasks passed, of 23	20	22	23	23	23
	\$ per priced task	\$0.306	\$0.399	\$0.330	\$0.435	\$0.458
	Minutes per task	6.9	10.6	10.2	15.4	14.5
◆ GPT-6 Sol	Combined score	67.62	80.60	82.83	85.94	86.82
	Tasks passed, of 23	4	13	15	18	19
	\$ per priced task	\$1.02	\$1.76	\$2.33	\$2.00	\$2.52
	Minutes per task	13.0	17.5	23.8	20.9	24.5
◇ GPT-5.6 Sol	Combined score	69.05	82.56	85.76	86.47	87.18
	Tasks passed, of 23	6	12	20	21	22
	\$ per priced task	\$1.36	\$1.72	\$2.05	\$1.61	\$1.77
	Minutes per task	9.8	11.1	12.0	10.6	11.6

Combined score averages judged runs, as on each model's own card. Tasks passed, cost and minutes cover all 23 runs per cell.

GPT-5.6 Sol max and GPT-6 Sol medium are each judged on 22 of 23 runs: one invalid judge probe on codecore in each; those runs are counted and priced.

GPT-6.1 Sol medium paddockcore has no valid Grok review (a changed quote on both attempts) and is scored from Muse alone.

Each model was judged in its own protocol run (GPT-6.1 Sol v3.18, GPT-6 Sol v3.17, GPT-5.6 Sol v3.7) with the same rubric and judges.

GPT-6.1 Sol ran on Codex CLI 0.159.0, GPT-6 on 0.155.0, GPT-5.6 on 0.153.4 or older.

List prices per million tokens, input and output: GPT-6.1 Sol \$2.00 and \$10.00; GPT-6 Sol \$2.00 and \$10.00;

GPT-5.6 Sol \$4.00 and \$20.00. Solver inference only; subscription bills differ.